

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job with SCD Type 2 This AWS Glue job implements a complete ETL pipeline for customer order analytics with Slowly Changing Dimension (SCD) Type 2 tracking using Apache Hudi.---

****High-Level Purpose****

Reads customer and order data from S3, cleans it, applies SCD Type 2 transformations, calculates aggregates, and writes results back to S3 in Hudi and Parquet formats while registering tables in the AWS Glue Catalog.---

****Step-by-Step Breakdown****

****1. Imports and Logging Setup****

- Imports PySpark, AWS Glue, and Boto3 libraries- Configures structured logging for job monitoring- Sets up type hints for better code clarity

****2. Context Initialization (`initialize\_spark\_contexts`)****

- Creates SparkContext, GlueContext, and SparkSession- Configures Spark for Hudi integration (Kryo serializer, adaptive execution)- Sets log level to WARN to reduce noise- **Maps to**: FR-INIT-001

****3. S3 Path Validation (`validate\_s3\_path`)****

- Parses S3 URIs (bucket + prefix)- Uses Boto3 to verify bucket accessibility via `head\_bucket`- Checks if prefix contains objects using `list\_objects\_v2`- Returns boolean indicating path validity- **Maps to**: FR-VALIDATE-001

****4. Safe Data Reading (`read\_data\_safe`)****

- Validates S3 path before reading- Reads CSV files with configurable schema enforcement- Normalizes column names to lowercase for consistency- Logs record counts and handles empty datasets- **Maps to**: FR-INGEST-001 (customers), FR-INGEST-002 (orders)

****5. Safe Data Writing (`write\_data\_safe`)****

- Validates bucket access before writing- Supports multiple formats (Parquet, Hudi)- Handles partitioning and write modes (overwrite/append)- Checks for empty DataFrames before writing- **Maps to**: FR-OUTPUT-001, FR-OUTPUT-002

****6. Data Cleaning (`clean\_data`)****

- **Step 1**: Removes rows with NULL values using `dropna()`- **Step 2**: Filters out rows containing 'Null' strings (case-insensitive)- **Step 3**: Removes duplicate records using `dropDuplicates()`- Logs counts at each stage for audit trail- **Maps to**: FR-CLEAN-001, FR-CLEAN-002

****7. SCD Type 2 Transformation (`apply\_scd\_type2`)****

Adds four tracking columns:- `isactive`: Boolean flag (True for current records)- `startdate`: Timestamp when record became active- `enddate`: Timestamp when record was superseded (NULL for active)- `opts`: Operation timestamp for versioning- `Maps to`: FR-SCD2-001

8. Hudi Table Writing (`write_hudi_table`)

Configures Hudi-specific options:- `Record key`: Primary key for upsert logic- `Precombine field`: Determines latest version (opts timestamp)- `Partition field`: Physical data partitioning (e.g., region)- `Table type`: COPY_ON_WRITE for read-optimized performance- Performs upsert operation to handle updates- `Maps to`: FR-SCD2-002

9. Aggregate Calculation (`calculate_customer_aggregate_spend`)

- Calculates `totalamount` = `priceperunit` × `qty`- Groups by order attributes (orderid, itemname, date)- Aggregates total amount and quantity per order- `Maps to`: FR-AGG-001

10. Glue Catalog Registration (`register_glue_catalog_table`)

- Creates Glue database if it doesn't exist- Registers table metadata using `saveAsTable()`- Enables SQL querying via Athena/Redshift Spectrum- `Maps to`: FR-CATALOG-001

11. Main Execution Flow (`main`)

Initialization Phase:

1. Retrieves job parameters using `getResolvedOptions` (input/output paths, database name)
2. Initializes Spark/Glue contexts
3. Defines schemas for customer and order data

Processing Phase:

4. **Step 1-2**: Reads customer and order CSV files from S3.
5. **Step 3-4**: Cleans both datasets (removes NULLs, 'Null' strings, duplicates)
6. **Step 5**: Applies SCD Type 2 columns to customer data
7. **Step 6**: Writes customer dimension to S3 using Hudi format with upsert
8. **Step 7**: Calculates order summary aggregates
9. **Step 8**: Writes order summary as partitioned Parquet (by date)
10. **Step 9**: Registers both tables in Glue Catalog

Completion Phase:

11. Commits Glue job and logs final status---

Key Technical Features

Error Handling

- All functions return Optional types or booleans- Comprehensive try-catch blocks with detailed logging- Graceful degradation (continues if Hudi write fails)

Data Quality

- Schema enforcement at read time- Multi-stage cleaning pipeline- Duplicate detection and removal

Performance Optimizations

- Adaptive query execution enabled- Configurable shuffle parallelism for Hudi (20 partitions)- Hive-style partitioning for efficient querying

Traceability

- Every function maps to specific functional requirements (FR-XXX-XXX)- Detailed logging at each processing stage- Record count tracking throughout pipeline---

****Data Flow Summary****

```
S3 (Raw CSV) → Read → Clean → Transform → Write (Hudi/Parquet) → Catalog
Registration ↓ ↓ ↓ ↓ ↓ ↓ Validate Schema Remove Add SCD2 Partition Enable Athena
Enforce Nulls Columns by Date Queries
```

****Configuration Requirements****

****Job Parameters:****- ``CUSTOMER_INPUT_PATH``: s3://adif-sdlc/sdlc_wizard/customerdata/-
``ORDER_INPUT_PATH``: s3://adif-sdlc/sdlc_wizard/orderdata/-
``ORDER_SUMMARY_OUTPUT_PATH``: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/-
``CUSTOMER_AGG_OUTPUT_PATH``: s3://adif-sdlc/analytics/customeraggregatespend/-
``GLUE_DATABASE``: Target database name for catalog registration****IAM Permissions Needed:****- S3
read/write access to specified buckets- Glue database/table create/update permissions- CloudWatch
Logs write access